

PYREADY

Python 과목평가 핵심 정리

시험 전 최종 점검을 위한 개념·문법·출력 해석 가이드

기본 문법	자료형	제어문	함수
자료구조	객체지향	예외 처리	답안 작성

이 자료의 활용 방식

각 항목은 정의, 대표 문법, 자주 혼동하는 지점, 출력 해석 기준을 함께 제시합니다. 예제를 눈으로만 읽지 말고 먼저 결과를 예상한 뒤 설명과 대조하면 효과적입니다.

수업에서 다룬 Python 기본 문법부터 예외 처리까지의 범위를 기준으로 구성했습니다.

01 답안 작성과 코드 해석

답안 형식

- 객관식: 코드의 실제 실행 순서를 끝까지 추적한 뒤 보기를 판단합니다. 익숙한 표현만 보고 결론을 먼저 정하지 않습니다.
- 단답형: 요구된 결과만 정확히 씁니다. 대소문자, 공백, 괄호, 쉼표, 따옴표 표시 여부를 출력 형태와 일치시킵니다.
- 서술형: 결론만 쓰지 않고 결과, 적용된 규칙, 실행 과정 또는 오류 원인을 연결해 설명합니다.

출력 문제의 기본 순서

① 초기 값과 자료형 확인 → ② 반복 범위와 조건 확인 → ③ 각 문장의 실행 후 상태 기록 → ④ print가 표현하는 실제 문자열 확인 → ⑤ 오류가 있다면 발생 지점과 이유 명시

서술형 답안의 안정적인 구조

구성	작성 내용	점검 질문
결과	출력값 또는 오류 이름	무엇이 출력되거나 발생하는가?
규칙	관련 문법·자료형의 성질	어떤 규칙이 적용되는가?
과정	값이 변하는 순서와 원인	왜 그 결과에 도달하는가?

짧은 답안이라도 문제에서 결과와 이유를 모두 요구한다면 두 요소를 빠뜨리지 않습니다. 오류 문제는 오류 이름만이 아니라, 어떤 인덱스·키·연산이 왜 유효하지 않은지까지 설명합니다.

02 연산자·형 변환

연산 결과를 결정하는 핵심

구분	핵심 규칙	대표 예
산술	/는 실수 나눗셈, //는 몫, %는 나머지, **는 거듭제곱	$7 // 3 \rightarrow 2$, $7 \% 3 \rightarrow 1$
비교	값의 크기·동등성을 비교해 bool을 반환	$3 == 3.0 \rightarrow \text{True}$
동등/동일	==는 값, is는 동일한 객체인지 비교	None 비교에는 is None 사용
논리	and/or는 필요할 때만 다음 피연산자를 평가하며 피연산자 자체를 반환할 수 있음	$0 \text{ or } 5 \rightarrow 5$, $'a' \text{ and } 3 \rightarrow 3$
멤버십	in은 포함 여부 검사. dict에서는 기본적으로 key를 검사	$'a' \text{ in } \{'a': 1\} \rightarrow \text{True}$

우선순위와 단축 평가

```
result = 2 + 3 * 4      # 14
value = 0 and (10 / 0)  # 0, 오른쪽은 평가하지 않음
name = '' or 'guest'    # 'guest'
```

괄호가 있으면 괄호를 먼저 계산합니다. 그다음 거듭제곱, 단항 연산, 곱셈·나눗셈, 덧셈·뺄셈, 비교, not, and, or의 흐름을 기준으로 해석합니다.

형 변환

표현	결과/주의
<code>int('12')</code>	12. 숫자 형태가 아니면 ValueError
<code>int(3.9)</code>	3. 소수점 아래를 버림
<code>float('3.5')</code>	3.5
<code>str(10)</code>	'10'
<code>bool(0), bool('')</code>	둘 다 False
<code>bool('0'), bool([0])</code>	비어 있지 않으므로 둘 다 True

자주 혼동하는 지점

실수는 이진 부동소수점으로 근사값을 저장할 수 있어 $0.1 + 0.2$ 가 0.3 과 정확히 같지 않을 수 있습니다. 또한 `is`를 값 비교의 대용으로 사용하지 않습니다.

03 문자열·시퀀스

시퀀스의 공통 성질

- 요소에 순서가 있으며 인덱싱, 슬라이싱, 길이 확인, 반복이 가능합니다.
- 대표 시퀀스는 str, list, tuple, range입니다. dict와 set은 시퀀스가 아닙니다.
- 순서가 있다는 말은 자동으로 정렬되어 있다는 뜻이 아닙니다.

표현	의미	예
s[i]	인덱스 i의 한 요소	'abc'[1] → 'b'
s[a:b]	a부터 b 직전까지	'abcde'[1:4] → 'bcd'
s[a:b:k]	간격 k로 슬라이싱	[0,1,2,3,4][::2] → [0,2,4]
s[::-1]	역순 복사	'abc'[::-1] → 'cba'

가변성과 불변성

자료형	변경 가능 여부	핵심
str	불변	문자 하나를 인덱스로 직접 바꿀 수 없음
list	가변	요소 추가·삭제·교체 가능
tuple	불변	고정된 묶음, 딕셔너리 key로 활용 가능
range	불변	정수 범위를 지연 표현
dict / set	가변	비시퀀스 자료형

튜플과 range의 기본 형태

```
one = (1,)      # 요소가 하나인 튜플
a, b = (10, 20) # 언패킹
range(2, 8, 2)  # 2, 4, 6 (끝값 8은 제외)
```

문자열 메서드 구분

find는 찾는 값이 없으면 -1을 반환하고, index는 ValueError를 발생시킵니다. isdecimal, isdigit, isnumeric은 인정하는 숫자 문자의 범위가 순서대로 넓어집니다.

04 제어문

조건문과 반복문

구문	판단 기준	핵심 점검
if / elif / else	위에서부터 조건을 평가	처음 참인 분기 하나만 실행
for	이터러블의 요소를 차례로 사용	반복 횟수보다 실제 요소 흐름 확인
while	조건이 참인 동안 반복	조건에 쓰인 값이 갱신되는지 확인
break	가장 가까운 반복문 종료	중첩 반복에서 어느 반복문을 끝내는지 확인
continue	현재 회차의 나머지를 건너뛴	갱신문까지 건너뛰어 무한 반복이 되는지 확인
pass	아무 동작도 하지 않는 자리 표시	흐름을 건너뛰거나 종료하지 않음

for-else와 유용한 반복 도구

표현	핵심
for ... else	반복이 break 없이 정상 종료될 때 else 실행
enumerate(xs, start=1)	인덱스와 요소를 함께 제공; start는 시작 인덱스
zip(a, b)	같은 위치의 요소를 묶으며 가장 짧은 이터러블에서 종료
map(func, xs)	함수를 각 요소에 적용하는 이터레이터를 반환
[expr for x in xs if cond]	반복과 조건을 이용해 새 리스트 생성

```

numbers = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
for i in range(len(numbers)):
    for j in range(len(numbers)):
        print(numbers[j][i], end=' ')
# 1 4 7 2 5 8 3 6 9

```

중첩 반복 해석

바깥 반복 변수 하나를 고정한 상태에서 안쪽 반복이 끝까지 도는 과정을 먼저 적습니다. numbers[j][i]처럼 인덱스 순서가 바뀌면 행이 아니라 열 단위로 접근할 수 있습니다.

05 함수·스코프

함수의 입력과 출력

개념	핵심
parameter	함수 정의에서 입력을 받는 변수
argument	함수 호출 시 전달하는 실제 값
return	함수를 즉시 끝내고 호출한 곳에 값을 전달
print	화면에 표시할 뿐 그 값 자체를 반환하지 않음
반환문 생략	함수는 None을 반환
여러 값 반환	쉼표로 나열한 값은 하나의 tuple로 묶여 반환

```
def pair(a, b=0):
    return a, b

result = pair(3)    # (3, 0)

def collect(*args, **kwargs):
    return args, kwargs
# args는 tuple, kwargs는 dict
```

인자 전달 규칙

- 위치 인자는 순서대로 대응하고, 키워드 인자는 매개변수 이름으로 대응합니다.
- 기본값 매개변수는 기본값이 없는 매개변수 뒤에 둡니다.
- 호출 시 일반 위치 인자는 키워드 인자보다 앞에 둡니다.
- *args는 남은 위치 인자를 튜플로, **kwargs는 남은 키워드 인자를 딕셔너리로 모읍니다.

이름 탐색: LEGB

순서	영역	설명
L	Local	현재 함수 내부
E	Enclosed	바깥 함수의 지역 영역
G	Global	모듈 전체 영역
B	Built-in	파이썬 내장 이름 영역

스코프 점검

이름을 사용할 때는 $L \rightarrow E \rightarrow G \rightarrow B$ 순서로 찾습니다. 함수 안에서 같은 이름에 값을 대입하면 기본적으로 지역 이름이 됩니다. list, sum 같은 내장 함수 이름을 변수로 덮어쓰지 않도록 주의합니다.

재귀 함수

함수가 자기 자신을 호출하는 구조입니다. 종료 조건(base case)이 없거나 도달하지 못하면 호출이 계속되어 오류가 발생합니다. 각 호출의 인자와 반환값을 단계별로 분리해 추적합니다.

06 자료구조와 메서드

리스트: 원본 변경 여부

표현	결과	반환값/주의
a.append(x)	x 하나를 끝에 추가	원본 변경, None 반환
a.extend(xs)	xs의 각 요소를 추가	문자열이면 문자 단위로 추가
a.sort()	원본을 정렬	None 반환
sorted(a)	정렬된 새 리스트 반환	원본 유지
a.reverse()	원본 순서를 뒤집음	None 반환
a.pop(i)	인덱스 i 요소 제거	제거한 요소 반환

복사와 별칭

```
a = [[1], [2]]
b = a      # 같은 객체를 가리키는 별칭
c = a[:]   # 바깥 리스트만 새로 만든 얇은 복사

a[0].append(9) # c[0]에서도 보임: 내부 리스트는 공유
```

딕셔너리

표현	핵심
d[key]	키가 없으면 KeyError
d.get(key)	키가 없으면 None; 기본값 지정 가능
d.keys()/values()/items()	키/값/(키, 값) 쌍을 제공
d.update(other)	기존 키는 덮어쓰고 새 키는 추가
d.setdefault(k, v)	키가 없을 때만 v를 저장하고 해당 값을 반환
d.pop(k)	키를 삭제하고 값을 반환; 키가 없으면 기본적으로 KeyError

06 자료구조와 메서드 (계속)

집합 set

항목	핵심
성질	중복을 허용하지 않으며 인덱싱할 수 없음
빈 집합	{ }가 아니라 set()으로 생성
합집합	$a \cup b$
교집합	$a \cap b$
차집합	$a - b$
remove(x)	x가 없으면 KeyError
discard(x)	x가 없어도 오류 없음

딕셔너리 key와 집합 원소

해시 가능한 값이 필요합니다. 일반적으로 불변 객체인 int, str, tuple은 사용할 수 있지만, 변경 가능한 list와 dict는 사용할 수 없습니다. 튜플 내부에 list가 포함되어 있다면 그 튜플도 해시 가능한 key가 될 수 없습니다.

메서드 문제의 첫 질문

이 메서드가 원본을 바꾸는가? 무엇을 반환하는가? 두 질문을 분리합니다. append, extend, sort, reverse처럼 원본을 바꾸는 리스트 메서드는 대개 None을 반환하므로 `result = a.sort()`의 result는 정렬된 리스트가 아니라 None입니다.

자료형별 빠른 비교

자료형	순서	변경	중복	주요 접근
list	있음	가능	허용	index
tuple	있음	불가	허용	index
dict	삽입 순서 유지	가능	key 중복 불가	key
set	인덱스 순서 없음	가능	불가	membership

07 객체지향

클래스와 인스턴스

개념	핵심
class	공통 속성과 동작을 정의하는 설계
instance	클래스를 호출해 만든 개별 객체
self	인스턴스 메서드를 호출한 인스턴스 자신
__init__	인스턴스 생성 과정에서 속성을 초기화
instance variable	인스턴스마다 따로 저장되는 속성
class variable	클래스와 인스턴스들이 공유해 참조하는 속성

```
class Animal:
    category = 'animal'

    def __init__(self, name):
        self.name = name

    def speak(self):
        return self.name

dog = Animal('dog')
```

메서드 구분

종류	첫 인자	주요 용도
인스턴스 메서드	self	인스턴스 상태 조회·변경
클래스 메서드	cls	@classmethod, 클래스 상태와 대안 생성 방식
정적 메서드	자동 전달 없음	@staticmethod, 클래스에 관련된 독립 기능

상속·오버라이딩·super

- 자식 클래스는 부모 클래스의 속성과 메서드를 물려받아 재사용할 수 있습니다.
- 자식 클래스가 같은 이름의 메서드를 다시 정의하면 오버라이딩됩니다.
- super()는 메서드 탐색 순서에 따라 다음 클래스의 구현을 호출합니다.
- 다중 상속에서는 MRO를 확인하여 속성과 메서드 탐색 순서를 판단합니다.
- __str__은 객체를 사람이 읽기 좋은 문자열로 표현할 때 사용됩니다.

속성 탐색

instance.attr을 찾을 때 먼저 인스턴스 영역을 확인하고, 없으면 클래스와 상속 계층을 탐색합니다. 인스턴스에 같은 이름을 대입하면 공유 클래스 변수를 바꾸는 대신 해당 인스턴스의 속성이 새로 생길 수 있습니다.

08 예외 처리

try 문 실행 흐름

블록	실행 조건
try	예외 발생 가능성이 있는 코드 실행
except	지정한 예외가 발생했을 때 실행
else	try에서 예외가 발생하지 않았을 때 실행
finally	예외 발생 여부와 관계없이 실행

```
try:
    value = int(text)
except ValueError as error:
    print("변환 실패")
else:
    print(value)
finally:
    print("처리 완료")
```

대표 예외

예외	대표 상황
ValueError	자료형은 허용되지만 값이 부적절함: int('abc')
TypeError	연산·호출에 맞지 않는 자료형 또는 인자 사용
IndexError	시퀀스의 유효 범위를 벗어난 인덱스
KeyError	딕셔너리에 없는 키를 []로 접근
ZeroDivisionError	0으로 나눔

except 순서

여러 예외를 구분해 처리할 때는 구체적인 예외를 먼저, 넓은 범위의 예외를 뒤에 둡니다. 하나의 처리 블록에서 여러 예외를 함께 잡을 때는 except (ValueError, TypeError):처럼 튜플로 작성합니다.

EAFP와 LBYL

EAFP는 일단 실행하고 예외를 처리하는 방식이며, LBYL은 실행 전에 조건을 검사하는 방식입니다. 상황에 맞는 방식을 선택하되, 지나치게 넓은 예외 처리는 실제 오류 원인을 가릴 수 있습니다.

09 보충 핵심 문법

변수·대입·언패킹

표현	핵심
<code>a = b = []</code>	a와 b가 같은 리스트 객체를 참조
<code>x, y = 1, 2</code>	오른쪽 값을 패킹한 뒤 각 변수에 언패킹
<code>x, y = y, x</code>	임시 변수 없이 두 값을 교환
<code>first, *rest = values</code>	나머지 요소는 rest에 list로 수집
<code>None</code>	값이 없음을 나타내는 단일 객체; 비교는 <code>is None</code> 권장

변수와 객체

변수는 값을 담는 상자라기보다 객체를 가리키는 이름으로 이해하는 편이 정확합니다. 가변 객체를 여러 변수가 함께 참조하면 한 경로에서의 변경이 다른 변수에서도 관찰됩니다.

문자열의 주요 메서드

표현	결과/주의
<code>text.split(sep)</code>	구분자를 기준으로 나눈 list 반환
<code>sep.join(items)</code>	문자열 요소들을 구분자로 연결
<code>text.strip()</code>	양 끝의 공백 또는 지정 문자를 제거한 새 문자열
<code>text.replace(old, new)</code>	치환된 새 문자열; 원본 문자열은 변경되지 않음
<code>text.count(x)</code>	겹치지 않는 등장 횟수
<code>text.startswith(x)</code>	지정 문자열로 시작하는지 bool 반환

```
words = ' red,blue '.strip().split(',')
result = ' / '.join(words)
# words: ['red', 'blue']
# result: 'red / blue'
```

리스트 메서드 보충

메서드	핵심
<code>insert(i, x)</code>	인덱스 i 앞에 x 삽입
<code>remove(x)</code>	처음 만나는 값 x 삭제; 없으면 ValueError
<code>pop(i)</code>	인덱스 i의 값을 삭제하고 반환; 기본은 마지막 요소
<code>index(x)</code>	처음 만나는 x의 인덱스; 없으면 ValueError
<code>count(x)</code>	x의 등장 횟수

09 보충 핵심 문법 (계속)

함수 호출과 스코프

문법	의미	주의
global name	함수에서 전역 이름에 대입	읽기만 할 때는 선언 불필요
func(*values)	이터러블을 위치 인자로 언패킹	개수와 매개변수 구조 확인
func(**options)	딕셔너리를 키워드 인자로 언패킹	key는 매개변수 이름과 대응
def f(a, b=0)	b에 기본값 지정	기본값 없는 매개변수를 앞에 배치

```
rate = 1
def change():
    global rate
    rate = 2

values = (10, 20)
options = {'sep': '-'}
print(*values, **options) # 10-20
```

모듈과 패키지

표현	사용 결과
import math	math 이름으로 모듈을 가져오며 math.sqrt(4)처럼 접근
import math as m	모듈에 별칭 m을 부여
from math import sqrt	sqrt 이름을 현재 영역으로 직접 가져옴
from math import sqrt as root	가져온 이름에 별칭 root를 부여
package.module	관련 모듈을 디렉터리 구조로 묶어 관리

이름 충돌 점검

from 모듈 import 이름 형식은 현재 영역에 이름을 직접 추가합니다. 이미 같은 이름의 변수나 함수가 있으면 덮어쓸 수 있으므로, 어떤 이름이 현재 스코프를 가리키는지 확인합니다.

컴프리헨션의 읽는 순서

```
squares = [x * x for x in range(5) if x % 2 == 0]
# 반복: x를 0부터 4까지 → 조건: 짝수만 → 표현식: x * x
# 결과: [0, 4, 16]

pairs = [(i, j) for i in range(2) for j in range(2)]
# (0, 0), (0, 1), (1, 0), (1, 1)
```

중첩 컴프리헨션은 일반 중첩 for 문과 같은 순서로 읽습니다. 결과 표현식은 앞에 있지만, 실제 추적은 첫 번째 for부터 시작해 뒤쪽 for와 조건으로 진행합니다.

09 보충 핵심 문법 (계속)

복사 수준 비교

방식	바깥 객체	내부 가변 객체
<code>b = a</code>	공유	공유
<code>b = a.copy()</code> 또는 <code>a[:]</code>	새로 생성	공유하는 얇은 복사
<code>b = deepcopy(a)</code>	새로 생성	재귀적으로 새로 복사

복사 문제의 관찰 지점

id 자체를 외우기보다 바깥 컨테이너와 내부 객체를 나누어 봅니다. 얇은 복사 후 `b.append(...)`는 바깥 리스트만 바꾸지만, `b[0].append(...)`는 공유된 내부 리스트를 바꿀 수 있습니다.

객체지향 보충

개념	핵심
namespace	이름과 객체의 대응을 저장하는 영역; 클래스와 인스턴스는 서로 다른 영역을 가짐
decorator	기존 함수를 다른 함수로 감싸 동작을 확장하는 문법; @표현식으로 적용
<code>__repr__</code>	개발자가 객체를 명확히 식별할 수 있는 표현에 사용
<code>__len__</code>	<code>len(instance)</code> 의 동작을 정의
<code>__eq__</code>	<code>instance == other</code> 의 비교 동작을 정의

오류 발생 시점

구분	발생 시점	예
SyntaxError	코드를 해석하는 단계에서 문법이 올바르지 않을 때	괄호 누락, 잘못된 함수 선언
NameError	실행 중 정의되지 않은 이름을 사용할 때	<code>print(unknown)</code>
실행 중 예외	해당 문장에 실제로 도달해 연산할 때	IndexError, KeyError, TypeError 등

실행 여부 판단

처리되지 않은 예외가 발생하면 그 문장의 정상 처리는 끝나지 않고 이후 문장도 실행되지 않습니다. try-except가 해당 예외를 처리하면 except 이후의 흐름으로 진행할 수 있습니다.

map과 지연 평가

Python 3의 map은 결과 전체를 즉시 리스트로 만들지 않고, 순회하면서 값을 생성하는 이터레이터를 반환합니다. 출력 형태가 필요하면 `list(map(...))`처럼 소비하며, 한 번 끝까지 소비한 이터레이터는 다시 값이 나오지 않을 수 있습니다.

10 자주 쓰이는 함수·메서드

내장 함수 빠른 참조

함수	반환/동작	점검할 부분
<code>print(*objects, sep=' ', end='\n')</code>	객체를 문자열로 표현해 출력	기본 <code>sep</code> 는 공백, <code>end</code> 는 줄바꿈
<code>input(prompt)</code>	입력받은 내용을 <code>str</code> 로 반환	숫자 계산 전 형 변환 필요
<code>len(obj)</code>	요소 개수를 <code>int</code> 로 반환	문자열은 문자 수, <code>dict</code> 는 <code>key</code> 수
<code>type(obj)</code>	객체의 자료형 반환	출력 형식은 <code><class 'int'></code> 등
<code>int / float / str / bool</code>	지정 자료형으로 변환	변환 불가능한 문자열은 <code>ValueError</code>
<code>range(start, stop, step)</code>	정수 범위를 표현하는 <code>range</code> 반환	<code>stop</code> 제외, <code>step</code> 은 0 불가
<code>enumerate(xs, start=0)</code>	(인덱스, 요소) 쌍을 제공	<code>start</code> 는 인덱스 시작값만 변경
<code>zip(*iterables)</code>	같은 위치의 요소를 <code>tuple</code> 로 묶음	가장 짧은 이터러블에서 종료
<code>map(func, iterable)</code>	함수 적용 결과를 생성하는 <code>iterator</code>	필요하면 <code>list</code> 로 변환; 소비 후 재사용 주의
<code>sorted(iterable, key=None, reverse=False)</code>	정렬된 새 <code>list</code> 반환	원본 유지, <code>key</code> 는 정렬 기준 함수

수치·판정 함수

함수	핵심	예
<code>sum(iterable, start=0)</code>	수치 요소의 합	<code>sum([1, 2, 3]) → 6</code>
<code>min(...)</code> / <code>max(...)</code>	가장 작은 값 / 큰 값	<code>key</code> 인자로 비교 기준 지정 가능
<code>abs(number)</code>	절댓값	<code>abs(-3) → 3</code>
<code>round(number, ndigits)</code>	지정 자릿수로 반올림	부동소수점 표현 특성 고려
<code>all(iterable)</code>	모든 요소가 <code>truthy</code> 이면 <code>True</code>	빈 이터러블은 <code>True</code>
<code>any(iterable)</code>	하나라도 <code>truthy</code> 이면 <code>True</code>	빈 이터러블은 <code>False</code>

함수 문제의 확인 순서

- ① 함수가 출력하는지 값을 반환하는지 확인 → ② 반환 자료형 확인 → ③ 원본 객체의 변경 여부 확인 → ④ `iterator`라면 언제 소비되는지 확인

10 자주 쓰이는 함수·메서드 (계속)

문자열 메서드

메서드	반환/동작	없거나 실패할 때
s.split(sep=None)	구분자로 나눈 list	sep 생략 시 연속 공백을 기준으로 분리
sep.join(strings)	문자열 요소를 연결한 str	요소가 str이 아니면 TypeError
s.strip(chars=None)	양 끝을 제거한 새 str	원본 문자열은 그대로
s.replace(old, new, count)	치환한 새 str	old가 없으면 원본과 같은 내용
s.find(sub)	처음 찾은 위치의 index	없으면 -1
s.index(sub)	처음 찾은 위치의 index	없으면 ValueError
s.count(sub)	등장 횟수	없으면 0
s.startswith(x) / endswith(x)	시작/끝 일치 여부 bool	tuple로 후보 여러 개 지정 가능
s.isdecimal / isdigit / isnumeric	숫자 문자 여부 bool	인정 범위가 순서대로 넓어짐
s.upper() / lower()	대/소문자로 바꾼 새 str	원본 문자열은 그대로

문자열 메서드의 공통 성질

문자열은 불변 객체이므로 메서드를 호출해도 원본 문자열 자체가 바뀌지 않습니다. 변환 결과를 계속 사용하려면 반환값을 변수에 다시 대입하거나 메서드 연결의 다음 입력으로 사용합니다.

```
text = ' Python,Java '
languages = text.strip().split(',')
result = ' / '.join(languages)
# languages: ['Python', 'Java']
# result: 'Python / Java'
```

find와 index

두 메서드는 찾았을 때 같은 index를 반환하지만, 값이 없을 때 find는 -1을 반환하고 index는 ValueError를 발생시킵니다. 조건 분기와 예외 처리 중 어떤 흐름이 필요한지에 따라 선택합니다.

10 자주 쓰이는 함수·메서드 (계속)

리스트 메서드

메서드	원본	반환값 / 실패
a.append(x)	x 하나를 끝에 추가	None
a.extend(iterable)	각 요소를 끝에 추가	None
a.insert(i, x)	index i 앞에 삽입	None
a.remove(x)	처음 만난 값 x 삭제	None / 없으면 ValueError
a.pop(i=-1)	index i 요소 삭제	삭제한 값 / 범위 밖이면 IndexError
a.sort(key=None, reverse=False)	원본 정렬	None
a.reverse()	원본 순서를 뒤집음	None
a.index(x)	변경 없음	처음 만난 index / 없으면 ValueError
a.count(x)	변경 없음	등장 횟수
a.copy()	변경 없음	얕은 복사한 새 list

딕셔너리 메서드

메서드	반환/동작	키가 없을 때
d.get(key, default=None)	값 조회	default 반환
d.keys() / values() / items()	key / 값 / (key, 값) view	빈 view
d.update(other)	key를 추가하거나 기존 값을 덮어씀	새 key 추가; None 반환
d.setdefault(key, default)	값 반환; 없을 때 저장	default를 저장하고 반환
d.pop(key, default)	key 삭제 후 값을 반환	default 생략 시 KeyError
d.clear()	모든 항목 삭제	None 반환

10 자주 쓰이는 함수·메서드 (계속)

집합 메서드와 연산

표현	원본/결과	값이 없을 때
s.add(x)	원본에 원소 하나 추가	이미 있으면 변화 없음
s.update(iterable)	원본에 여러 원소 추가	중복은 저장하지 않음
s.remove(x)	원본에서 x 삭제	KeyError
s.discard(x)	원본에서 x 삭제	오류 없음
s.pop()	임의의 원소를 삭제하고 반환	빈 set이면 KeyError
a b / a.union(b)	합집합을 새 set으로 반환	원본 유지
a & b / a.intersection(b)	교집합을 새 set으로 반환	원본 유지
a - b / a.difference(b)	차집합을 새 set으로 반환	원본 유지

원본 변경 여부 최종 비교

원본을 바꾸고 주로 None 반환	새 결과를 반환하고 원본 유지
list.append, extend, insert	sorted(iterable)
list.sort, reverse, remove	str의 split, strip, replace, upper, lower
dict.update, clear	dict.get 및 keys/values/items view 조회
set.add, update, remove, discard	set의 , &, - 연산 결과

빠른 판별

메서드 호출 결과를 변수에 저장하는 코드에서는 반환값을 먼저 확인합니다. `result = numbers.sort()`의 `result`는 정렬된 리스트가 아니라 `None`이며, 정렬된 원본은 `numbers`입니다. 반대로 `sorted(numbers)`는 새 리스트를 반환합니다.

메서드 호출 전 공통 점검

- 호출 주체의 자료형이 해당 메서드를 제공하는가?
- 메서드가 원본을 변경하는가, 새 객체를 반환하는가?
- 반환값을 변수에 저장했을 때 실제로 무엇이 들어가는가?
- 찾는 값이나 key가 없을 때 기본값을 반환하는가, 예외를 발생시키는가?

11 최종 점검표

혼동하기 쉬운 쌍

A	B	구분 기준
==	is	값의 동등성 / 객체의 동일성
/	//	실수 나눗셈 / 몫
find	index	실패 시 -1 / ValueError
append	extend	객체 하나 추가 / 이터러블의 요소별 추가
sort	sorted	원본 변경 / 새 객체 반환
remove	discard	set에서 부재 시 KeyError / 오류 없음
d[key]	d.get(key)	부재 시 KeyError / None 또는 기본값
print	return	화면 출력 / 호출 지점에 값 전달
break	continue	반복 종료 / 현재 회차 건너뛰기
class 변수	instance 변수	공유 참조 / 객체별 상태

코드를 보기 전에 확인할 질문

- 현재 값의 자료형은 무엇인가? 가변 객체인가?
- 연산자와 괄호의 평가 순서는 무엇인가? 단축 평가가 일어나는가?
- 슬라이싱의 끝 인덱스가 제외되는가? step 방향과 시작·끝이 맞는가?
- 반복문의 각 회차에서 어떤 값이 바뀌며 break/continue가 어디에 적용되는가?
- 함수는 무엇을 출력하고 무엇을 반환하는가? 이름은 어느 스코프에서 찾는가?
- 메서드는 원본을 변경하는가? 반환값은 무엇인가?
- 인스턴스 속성인가 클래스 속성인가? 상속 탐색 순서는 무엇인가?
- 오류는 정확히 어느 문장에서 발생하며 그 뒤 코드는 실행되는가?

마지막 학습 권장

각 비교 항목을 자신의 말로 한 문장씩 설명하고, 짧은 코드를 직접 작성해 예상 결과와 실제 결과를 대조합니다. 틀린 문제는 정답만 확인하지 말고 적용한 규칙과 잘못 판단한 지점을 함께 기록합니다.

정확한 개념 구분과 실행 순서 추적이 안정적인 답안의 기반입니다.